

Arquitectura y Organización del Computador

Clase pre-parcial

Segundo cuatrimestre 2026

DC-UBA

Objetivos de la clase de hoy:

- Repasar *offsets* y alineamientos de estructuras
- Repasar el cómo recorrer estructuras
- Repasar llamados a funciones
- Repasar aritmética de punteros

Aclaraciones



Enunciado

Una poderosa red social de Finlandia nos contrató para mejorar su código. En la primer parte del proyecto, la idea es mejorar la performance actual de los algoritmos, por lo cual nos pidieron que realicemos la implementación en ASM. Esta empresa se llama Tuitter. Dentro de la app hay usuarios, que pueden seguirse unos a otros. Cada uno puede publicar mensajes que serán visibles para todos sus seguidores.

Estructuras

1)

```
typedef struct {  
    char mensaje[140];  
    uint16_t favoritos;  
    uint16_t retuits;  
    uint32_t id_autor;  
} tuit_t;
```

Estructuras

1)

```
typedef struct {  
    char mensaje[140];  
    uint16_t favoritos;  
    uint16_t retuits;  
    uint32_t id_autor;  
} tuit_t;
```

2)

```
typedef struct {  
    publicacion_t* next;  
    tuit_t* value;  
} publicacion_t;
```

Estructuras

1)

```
typedef struct {  
    char mensaje[140];  
    uint16_t favoritos;  
    uint16_t retuits;  
    uint32_t id_autor;  
} tuit_t;
```

3)

```
typedef struct {  
    publicacion_t* first;  
} feed_t;
```

2)

```
typedef struct {  
    publicacion_t* next;  
    tuit_t* value;  
} publicacion_t;
```

Estructuras

1)

```
typedef struct {  
    char mensaje[140];  
    uint16_t favoritos;  
    uint16_t retuits;  
    uint32_t id_autor;  
} tuit_t;
```

3)

```
typedef struct {  
    publicacion_t* first;  
} feed_t;
```

2)

```
typedef struct {  
    publicacion_t* next;  
    tuit_t* value;  
} publicacion_t;
```

4)

```
typedef struct {  
    feed_t* feed;  
    usuario_t** seguidores;  
    uint32_t cantSeguidores;  
    usuario_t** seguidos;  
    uint32_t cantSeguidos;  
    usuario_t** bloqueados;  
    uint32_t cantBloqueados;  
    uint32_t id;  
} usuario_t;
```

Tuit

```
typedef struct {  
    char mensaje[140];  
    uint16_t favoritos;  
    uint16_t retuits;  
    uint32_t id_autor;  
} tuit_t;
```

tuit_t

mensaje[140]

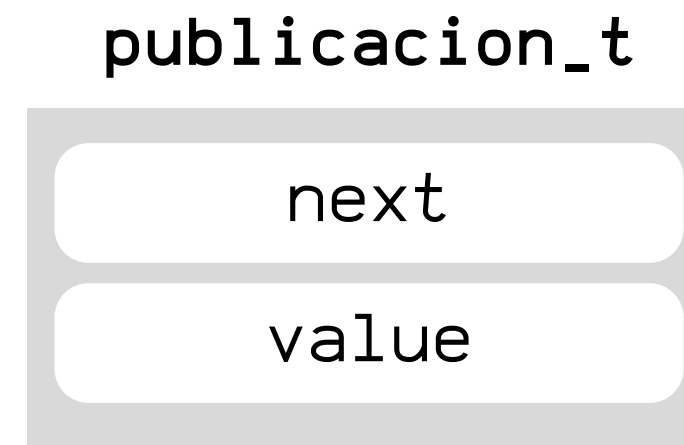
favoritos

retuits

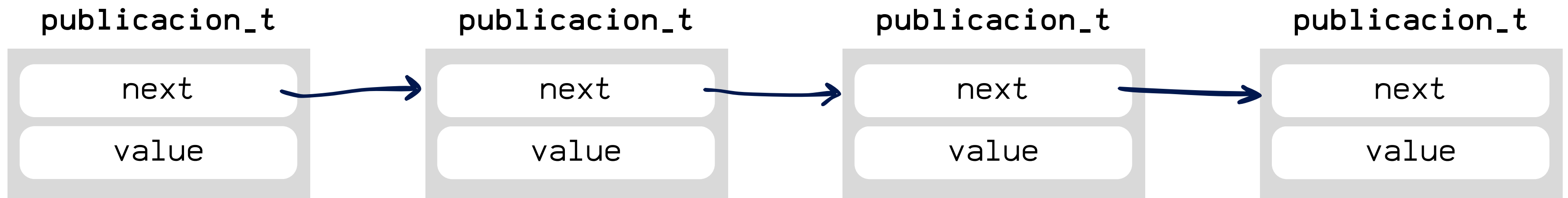
id_autor

Publicacion

```
typedef struct {  
    publicacion_t* next;  
    tuit_t* value;  
} publicacion_t;
```

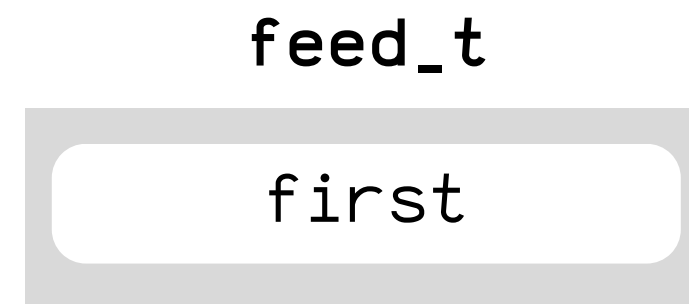


Publicaciones como listas enlazadas

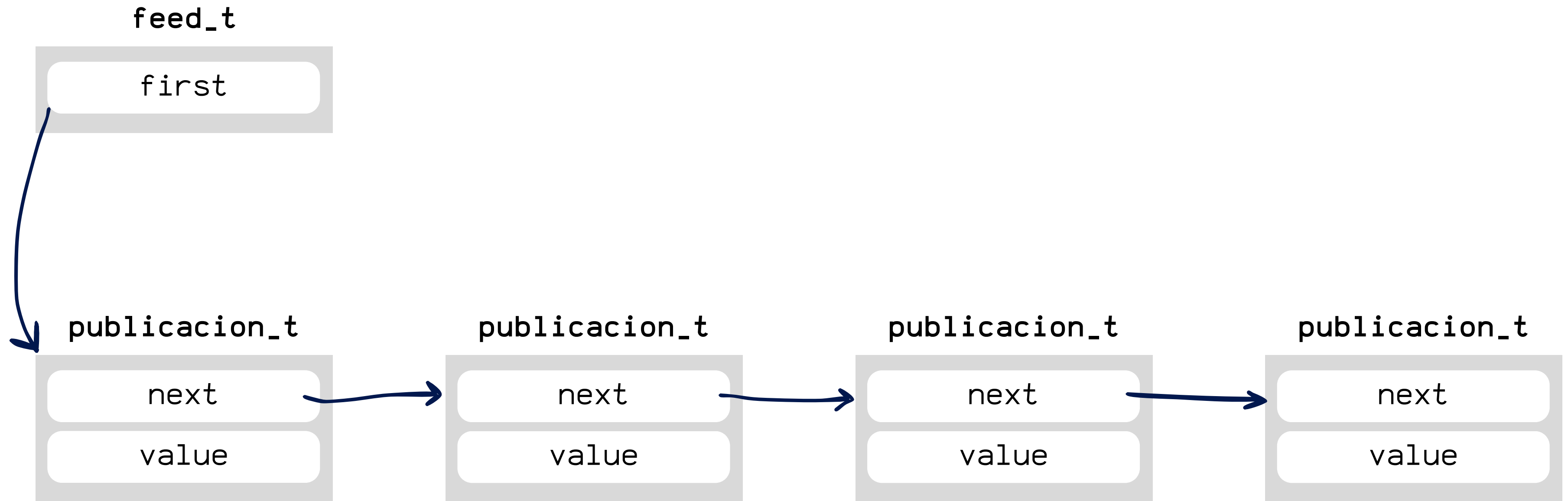


Feed

```
typedef struct {  
    publicacion_t* first;  
} feed_t;
```



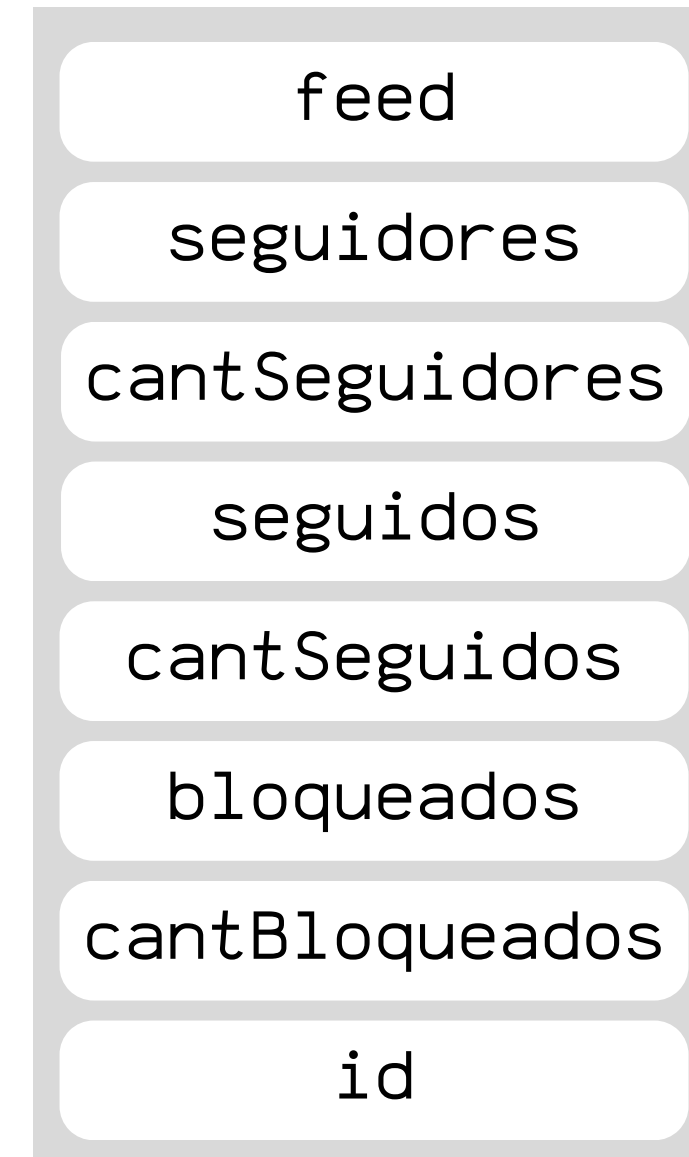
Punto de entrada a listas enlazadas



Publicacion

```
typedef struct {  
    feed_t* feed;  
    usuario_t** seguidores;  
    uint32_t cantSeguidores;  
    usuario_t** seguidos;  
    uint32_t cantSeguidos;  
    usuario_t** bloqueados;  
    uint32_t cantBloqueados;  
    uint32_t id;  
} usuario_t;
```

usuario_t



Ejercicio 1

Una de las funciones básicas es **publicar** tuits. Para ello, se solicita implementar la función:

```
tuit_t* publicar(char* mensaje, usuario_t* usuario)
```

que, tomando un mensaje y un usuario, debe armar un tuit y **agregarlo** como una **publicación** al **principio** de su propio feed y de los **feeds de sus seguidores**.

- El tuit **deberá ser uno solo**, pero las publicaciones son **propias de cada feed**.
- Al crear el tuit, el mensaje **debe ser clonado**.

Consejo: armar una función auxiliar que, dado un tuit y un feed, crea la publicación con el tuit y la agregue al comienzo del feed.

Ejercicio 1

```
tuit_t* publicar(char* mensaje, usuario_t* usuario)
```

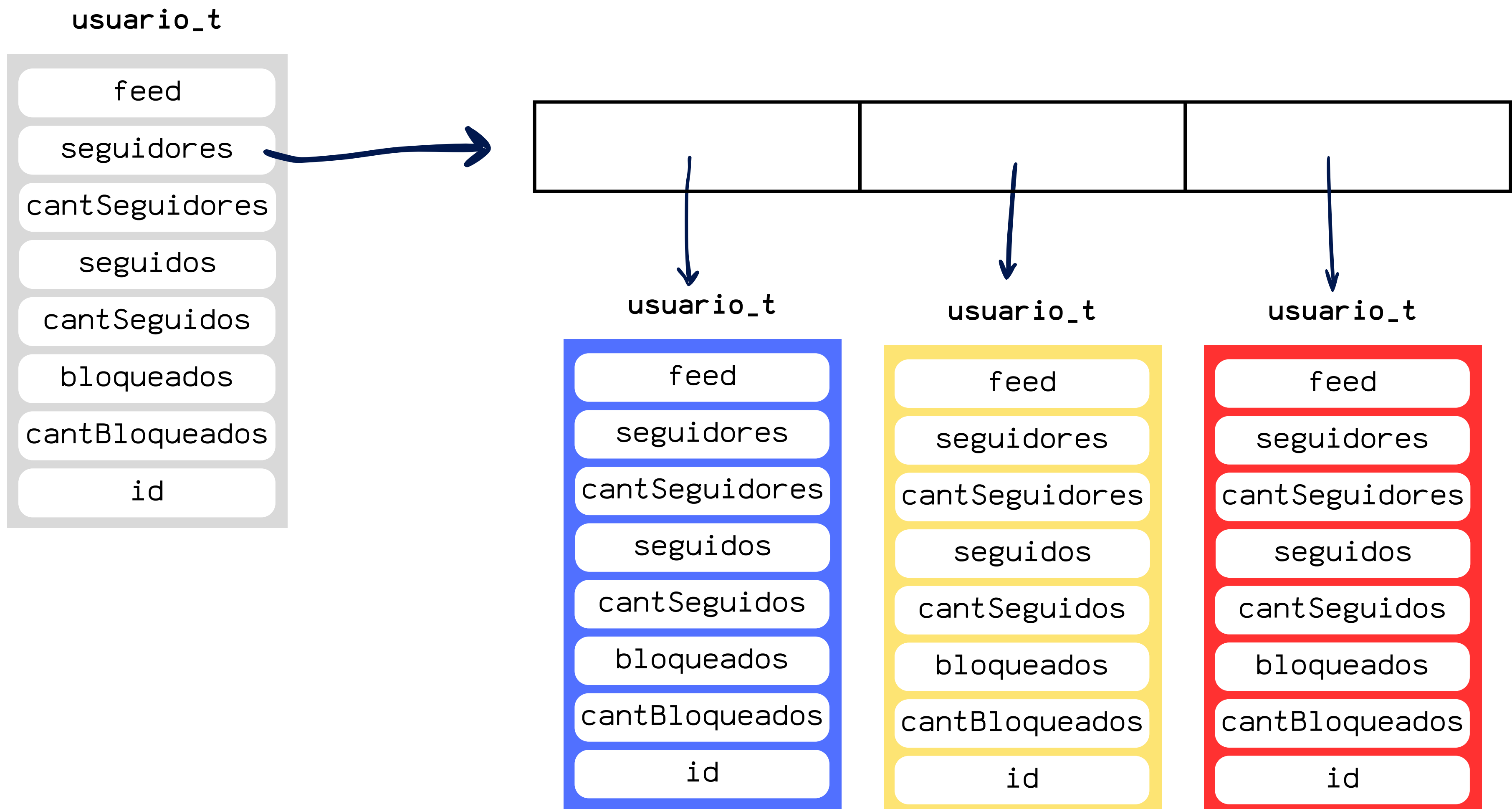
- Armar un tuit
- Agregarlo como una **publicación** al **principio** de su propio feed
- Agregarlo como publicación al principio de los **feeds de sus seguidores**
- Devolver el tuit

Ejercicio 1 - Pasos generales de la Función

```
tuit_t* publicar(char* mensaje, usuario_t* usuario)
```

- Armar un tuit
 - Pedir memoria para el tuit
 - Completar los campos que correspondan de la estructura
 - **Copiar** el mensaje
- Agregarlo como una **publicación** al **principio** de su propio feed
 - Acceder al feed del usuario
 - Invocar a la funcion auxiliar
- Agregarlo como publicación al principio de los **feeds de sus seguidores**
 - Acceder a la lista de seguidores del usuario y para cada uno:
 - Acceder al feed del seguidor
 - Invocar a la función auxiliar
- Devolver el tuit

Punto de entrada a lista de punteros a seguidores



Ejercicio 1 - Pasos generales de la Auxiliar

```
void* agregar_publicacion_feed(tuit_t* tuit, feed_t* feed)
```

- Armar la publicación
- Agregar la **publicación** al **principio** del feed

Ejercicio 1 - Pasos generales de la Auxiliar

```
void* agregar_publicacion_feed(tuit_t* tuit, feed_t* feed)
```

- Armar la publicación
 - Pedir memoria para la publicación
 - Completar los campos que correspondan
 - Poner el puntero al tuit
 - Poner como next el puntero que actualmente tiene en first el feed
- Agregar la publicación al principio del feed
 - Asignar en el campo first del feed el puntero a la nueva publicación

Ejercicio 2

Para mejorar la experiencia de los usuarios, queremos hacer que puedan bloquear a los usuarios que les generen incomodidad. Para ello, se solicita implementar la función:

```
void bloquearUsuario(usuario* usuario, usuario_t* usuarioABloquear)
```

que, tomando dos usuarios, bloquee el acceso del segundo a los tuits del primero y viceversa. Para esto, hay que:

- Agregar al usuario bloqueado al final del arreglo de usuarios bloqueados
- **Borrar** todas las publicaciones del feed del usuario bloqueador que contengan tuits del usuario bloqueado
- **Borrar** todas las publicaciones del feed del usuario bloqueado que contengan tuits del usuario bloqueador

Ejercicio 2: Algunas consideraciones

- Al borrar una publicación, procurar NO eliminar el tuit.
- Se puede asumir que el arreglo de usuarios bloqueados posee suficiente espacio libre para poder agregar elementos.

Consejo: armar una función auxiliar que tome un feed y un usuario, y borre todas las publicaciones del feed que contengan tuits del usuario pasado por parámetro.

Ejercicio 2: Charlando la problemática

```
void bloquearUsuario(usuario* usuario, usuario_t* usuarioABloquear)
```

Por parámetro recibimos dos usuarios, y sabemos que la estructura de cada usuario viene dada por la siguiente forma:

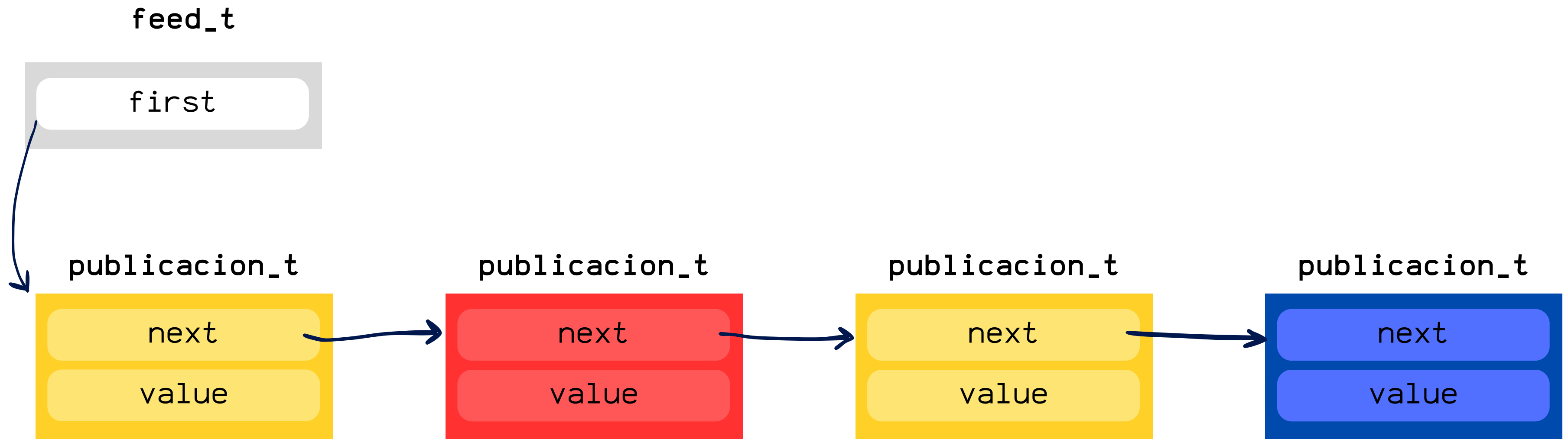
```
typedef struct {  
    feed_t* feed;  
    usuario_t** seguidores;  
    uint32_t cantSeguidores;  
    usuario_t** seguidos;  
    uint32_t cantSeguidos;  
    usuario_t** bloqueados;  
    uint32_t cantBloqueados;  
    uint32_t id;  
} usuario_t;
```

- Tenemos acceso al feed de cada uno
- Para el usuario **bloqueador** (usuario), podemos usar su arreglo bloqueados para agregar usuarioABloquear al final del mismo.
- Sabemos en dónde debemos colocar la variable usuarioABloquear puesto que tenemos la variable cantBloqueados
- Finalmente, tenemos acceso al *feed* de cada usuario.

Ejercicio 2: Diagramando ideas

```
void bloquearUsuario(usuario* usuario, usuario_t* usuarioABloquear)
```

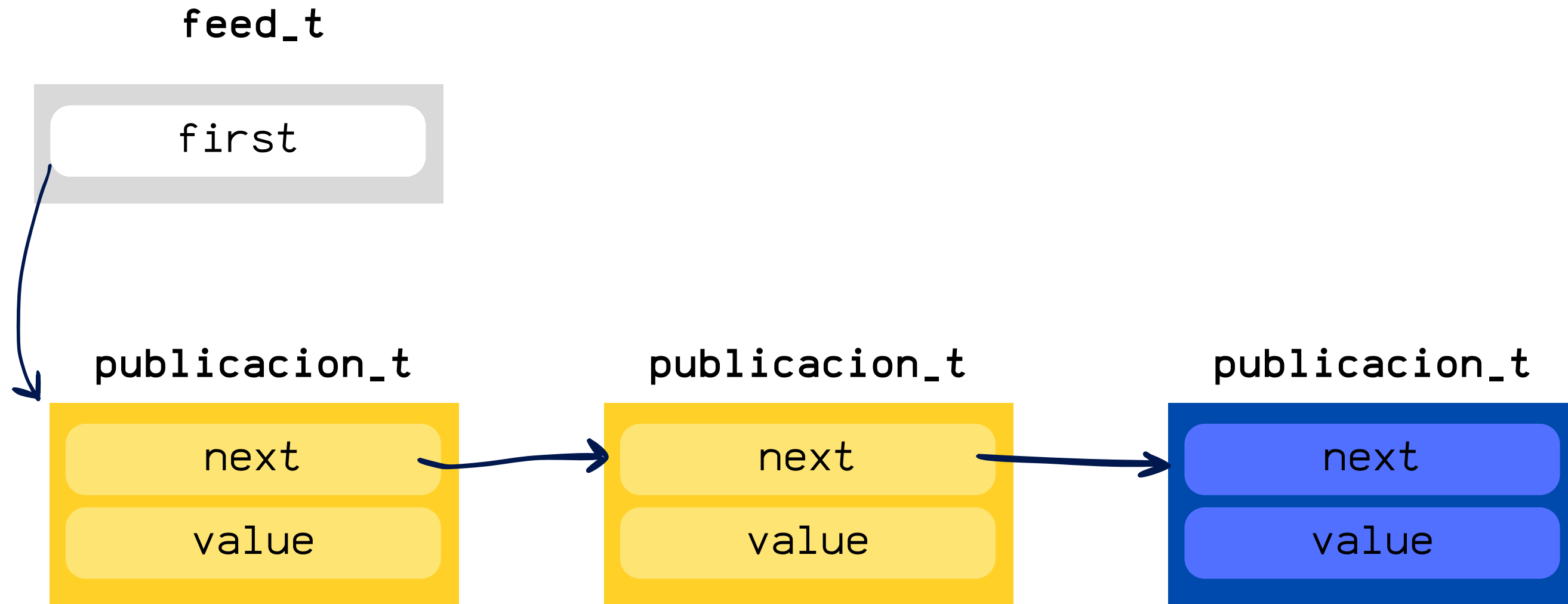
Supongamos que el usuario **amarillo** bloquea al usuario **rojo**, y el feed del usuario amarillo luce así:



Ejercicio 2: Diagramando ideas

```
void bloquearUsuario(usuario* usuario, usuario_t* usuarioABloquear)
```

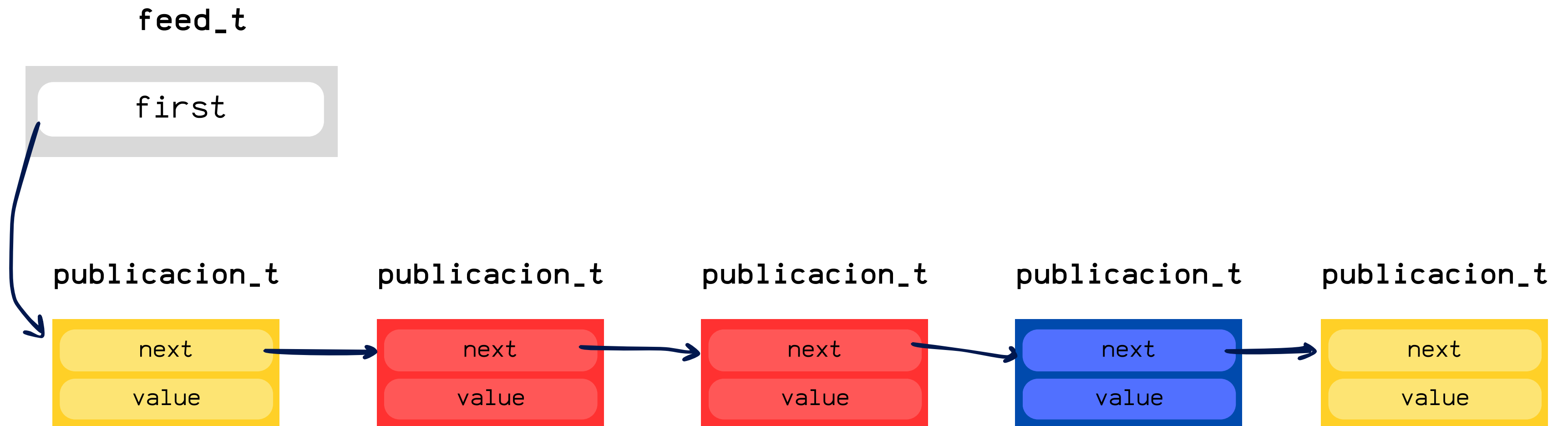
Esperamos que luego de el *feed* del usuario amarillo luzca de la siguiente forma:



Ejercicio 2: Diagramando ideas

```
void bloquearUsuario(usuario* usuario, usuario_t* usuarioABloquear)
```

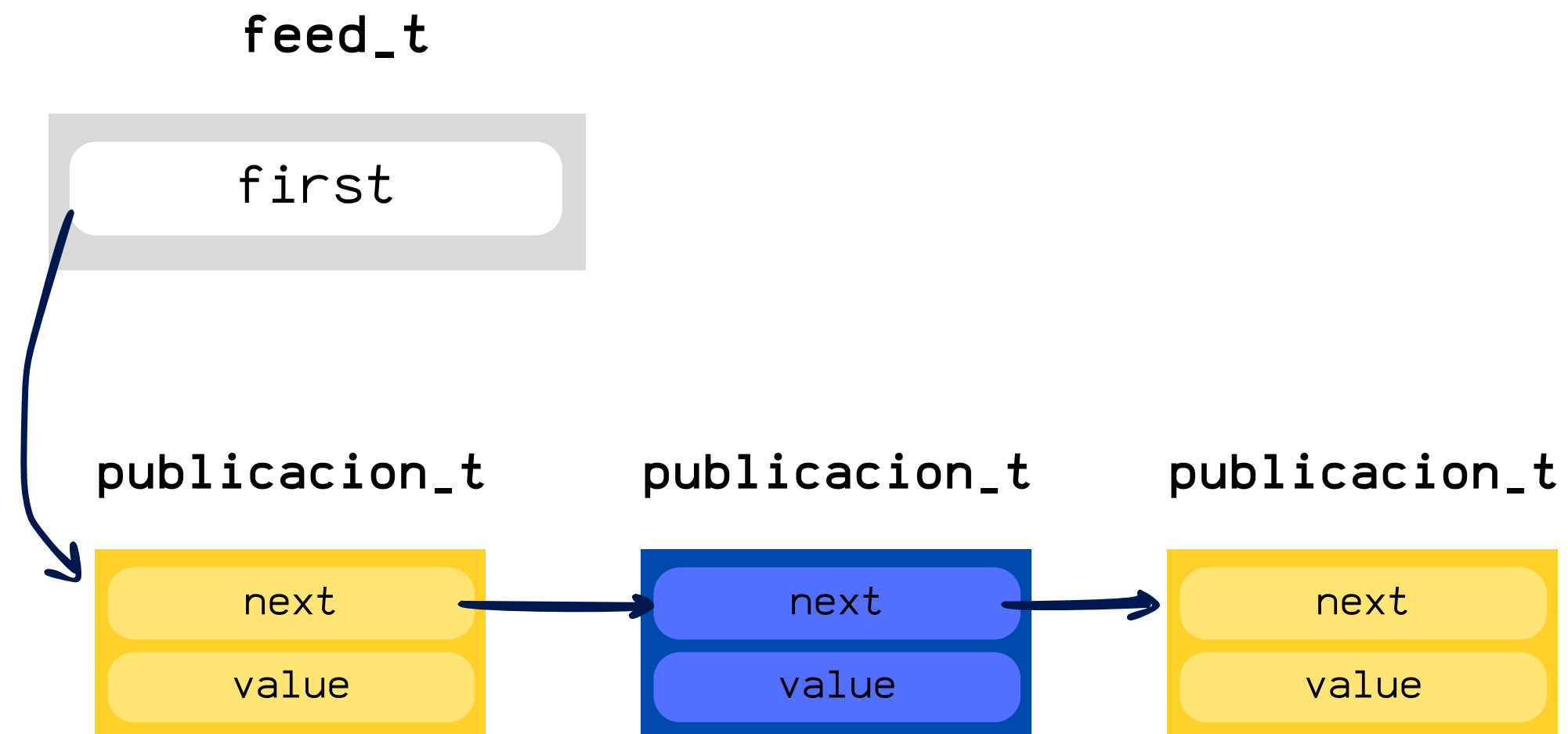
Mismo caso pero con otro *feed*:



Ejercicio 2: Diagramando ideas

```
void bloquearUsuario(usuario* usuario, usuario_t* usuarioABloquear)
```

Lo que esperamos obtener:



Ejercicio 2: Diagramando ideas

```
void bloquearUsuario(usuario* usuario, usuario_t* usuarioABloquear)
```

- En los casos anteriores, como el usuario **amarillo bloqueó** al usuario **rojo**, vimos que al usuario amarillo le **desaparecieron las publicaciones del usuario rojo**.
- De igual forma, al usuario rojo le **deben desaparecer las publicaciones amarillas**.

`eliminar_publicaciones_del_usuario`

Estos punteos denotan un **comportamiento equivalente**. Podemos escribir una función titulada `eliminar_publicaciones_del_usuario` para evitar tener que escribir la lógica de eliminar publicaciones dos veces.

Ejercicio 2: Diagramando ideas

```
void bloquearUsuario(usuario* usuario, usuario_t* usuarioABloquear)
```

- En los casos anteriores, como el usuario **amarillo bloqueó** al usuario **rojo**, vimos que al usuario amarillo le **desaparecieron las publicaciones del usuario rojo**.
- De igual forma, al usuario rojo le **deben desaparecer las publicaciones amarillas**.

Estos punteos denotan un **comportamiento equivalente**. Podemos escribir una función titulada `eliminar_publicaciones_del_usuario` para evitar tener que escribir la lógica de eliminar publicaciones dos veces.

Ejercicio 2: Manos a la obra

```
void bloquearUsuario(usuario* usuario, usuario_t* usuarioABloquear)
```

1. Agregar a **usuarioABloquear** al final del arreglo **bloqueados** de **usuario**.
2. Eliminar las publicaciones de **usuarioABloquear** del *feed* de **usuario**.
3. Eliminar las publicaciones de **usuario** del *feed* del **usuarioABloquear**.

Ejercicio 2: Algunas consideraciones

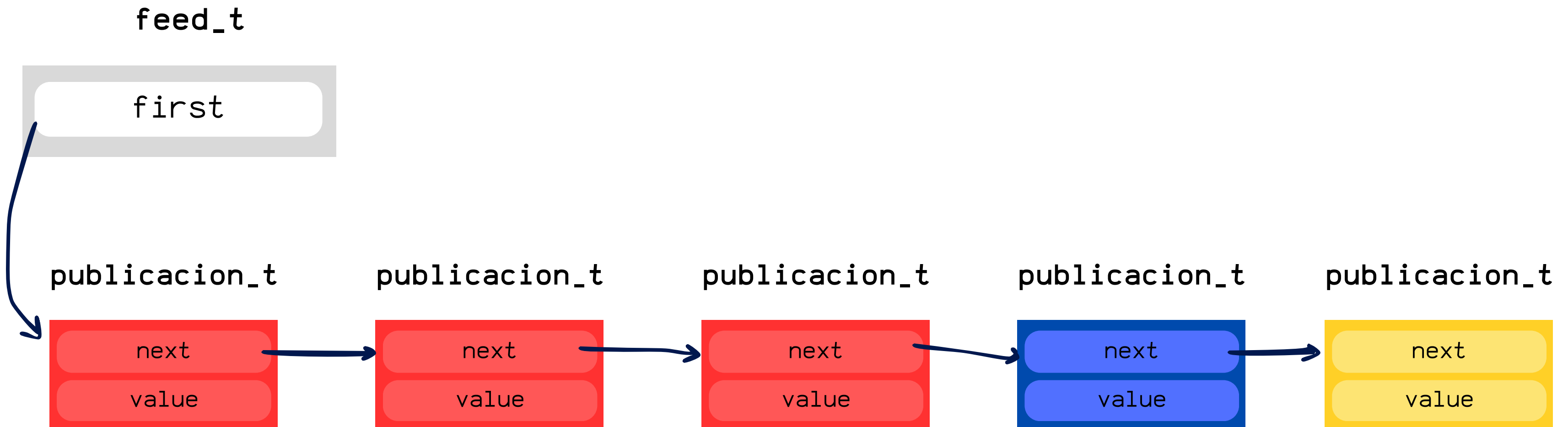
```
void eliminar_publicaciones_del_usuario(feed_t* feed, uint32_t id_usuario)
```

1. El *feed* debe terminar apuntando a una publicación que **NO** pertenezca al usuario que se pasó por parámetro:

Ejercicio 2: Algunas consideraciones

```
void eliminar_publicaciones_del_usuario(feed_t* feed, uint32_t id_usuario)
```

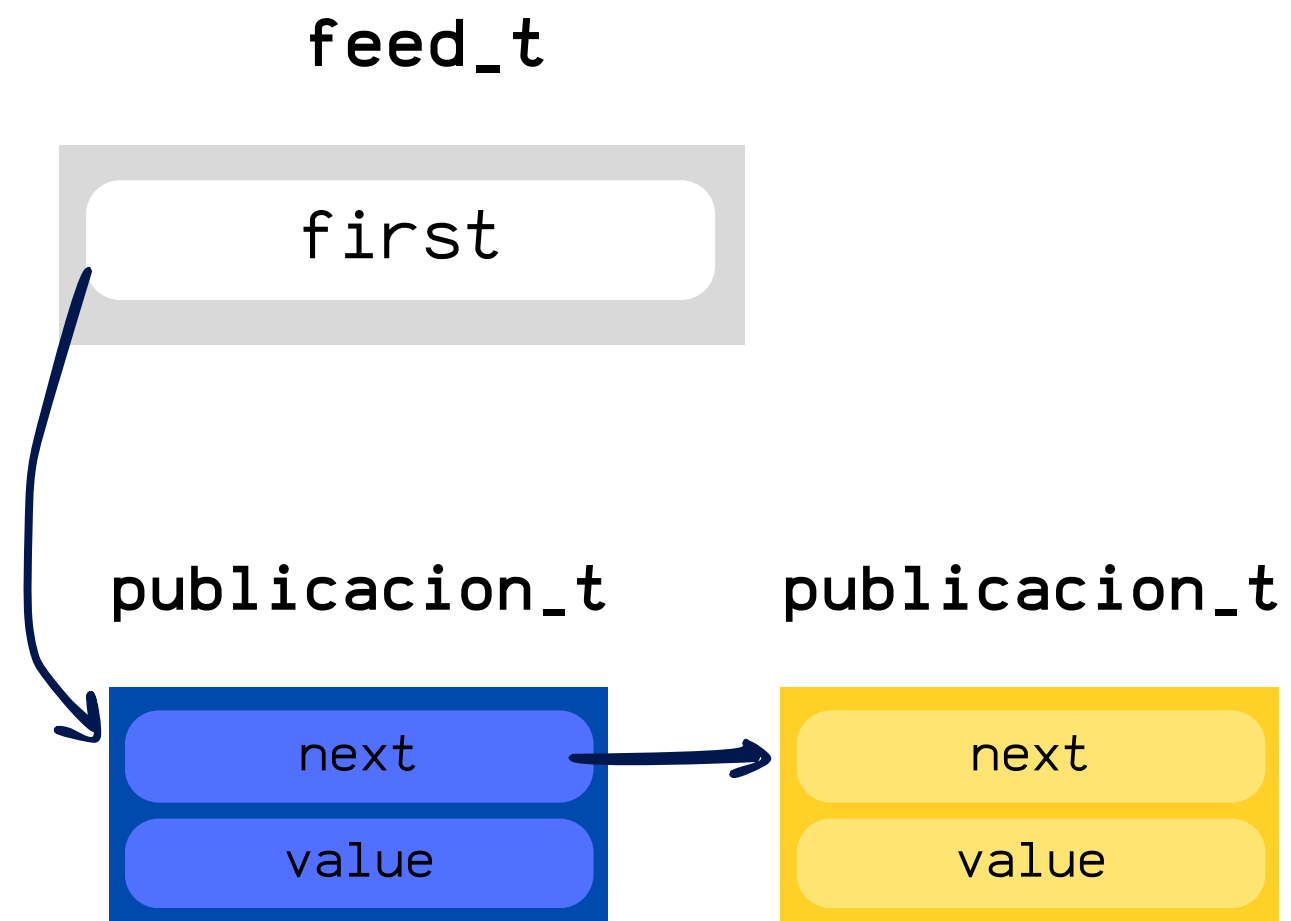
1. El *feed* debe terminar apuntando a una publicación que **NO** pertenezca al usuario que se pasó por parámetro:



Ejercicio 2: Algunas consideraciones

```
void eliminar_publicaciones_del_usuario(feed_t* feed, uint32_t id_usuario)
```

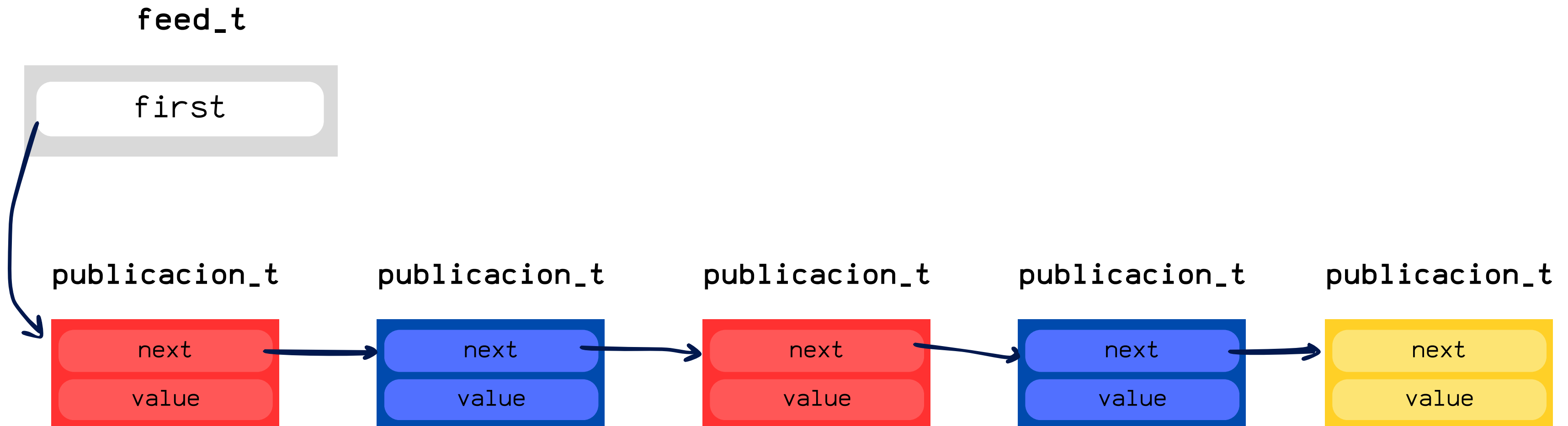
1. El *feed* debe terminar apuntando a una publicación que **NO** pertenezca al usuario que se pasó por parámetro:



Ejercicio 2: Algunas consideraciones

```
void eliminar_publicaciones_del_usuario(feed_t* feed, uint32_t id_usuario)
```

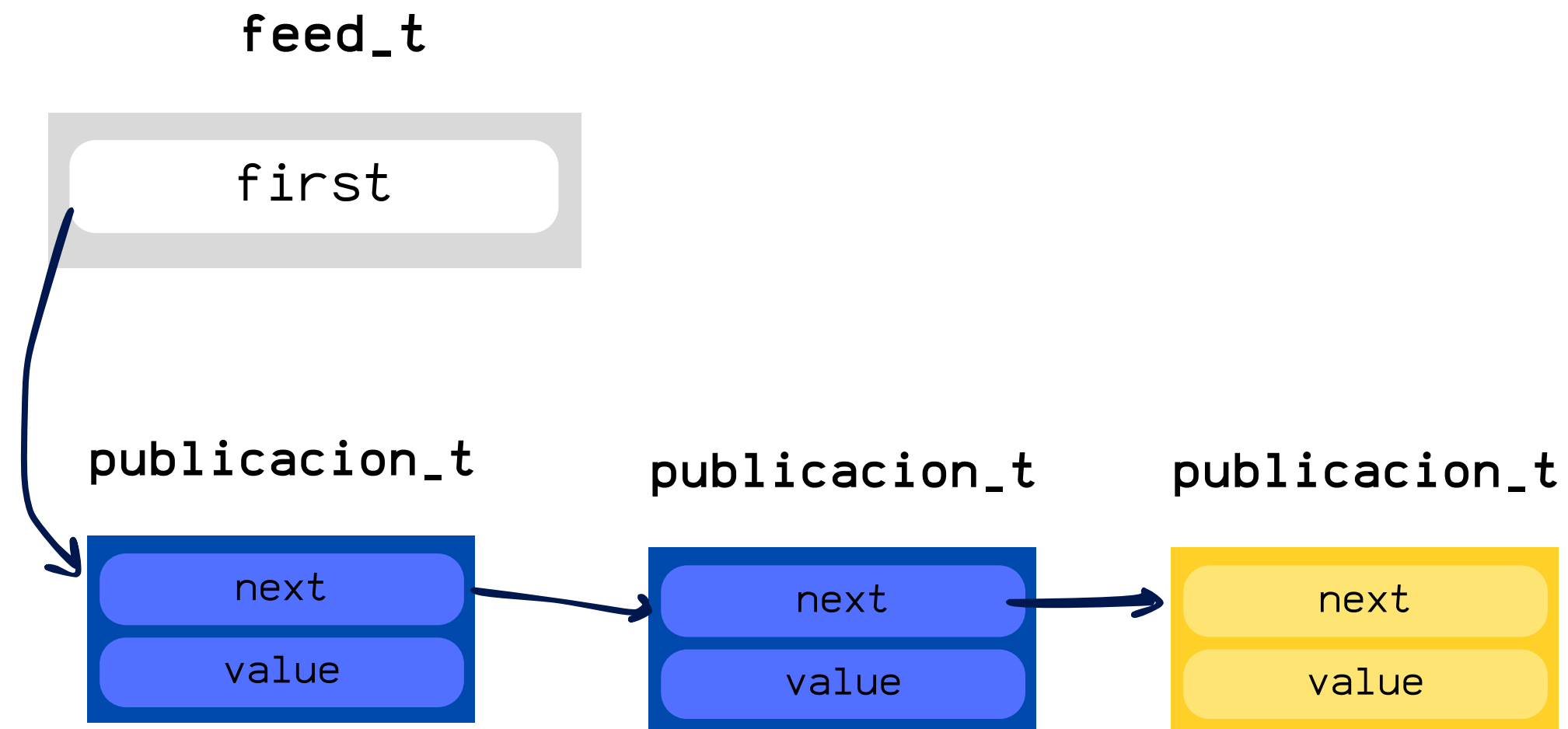
2. El *feed* debe apuntar **a la primera** publicación que NO SEA del usuario que estamos bloqueando.
3. Hay que mantener la lista enlazada enlazada, valga la redundancia.



Ejercicio 2: Algunas consideraciones

```
void eliminar_publicaciones_del_usuario(feed_t* feed, uint32_t id_usuario)
```

2. El *feed* debe apuntar **a la primera** publicación que NO SEA del usuario que estamos bloqueando.



Ejercicio 3

Para incentivar la interacción entre todos los usuarios de la red, queremos implementar una sección "Trending Topic". Para lograr esto, vamos a seleccionar los tuits sobresalientes de un usuario.

```
tuit_t** trendingTopic(usuario* usuario, uint8_t (*esTuitSobresaliente)(tuit_t*))
```

que recibe un usuario y una función que determina si un tuit es sobresaliente o no (devuelve 0 si no, 1 si sí).

- La función trendingTopic recorrerá el feed del usuario y devolverá un arreglo que contenga punteros a los tuits sobresalientes **hechos por el usuario**.
- El arreglo devuelto debe **terminar** con un elemento NULL para marcar el final, a menos que no haya tuits sobresalientes, en cuyo caso no se debe devolver un arreglo, sino un puntero a NULL.

Ejercicio 3: Consideraciones

```
tuit_t** trendingTopic(usuario* usuario, uint8_t (*esTuitSobresaliente)(tuit_t*))
```

- Procurar no devolver tuits de otros usuarios.

- No se deben copiar los tuits

Consejo: armar una función auxiliar que cuente la cantidad de tuits sobresalientes del feed de un usuario.

¿Consultas?